

# HTTP: The Hypertext Transfer Protocol

CEN 223 - Internet Communication

Assoc. Prof. Dr. Fatih ABUT

Department of Computer Engineering

Çukurova University

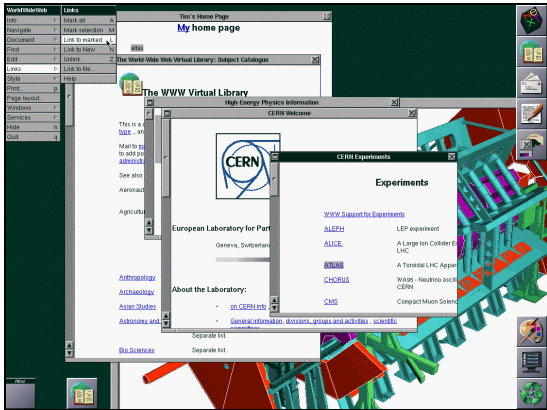
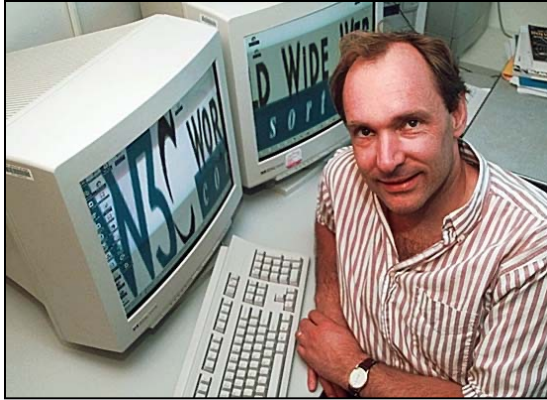
# HTTP Overview

## Design Criteria:

- Simplicity
- Efficiency
- Performance

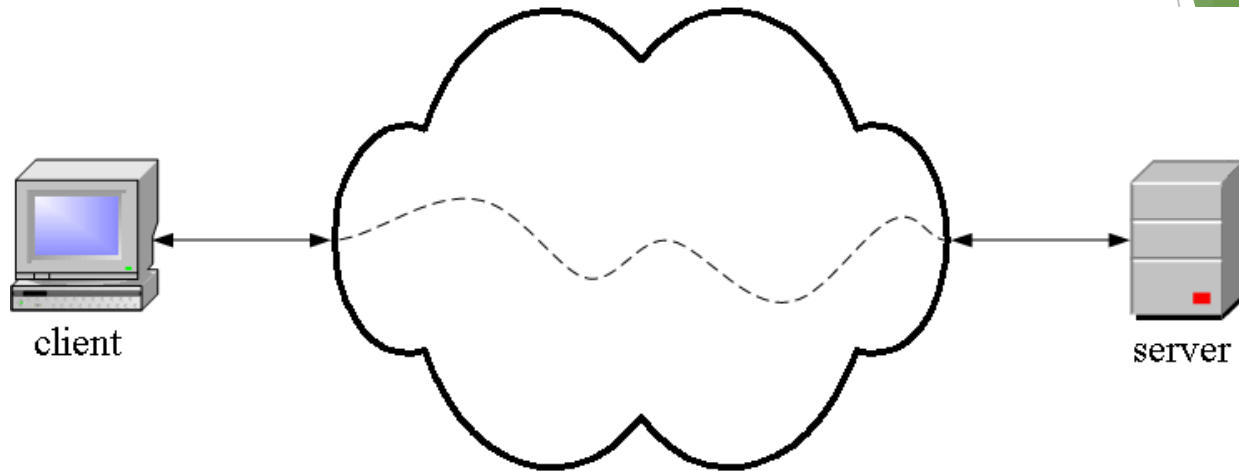
- HTTP is based on the TCP protocol.
- The web server uses the TCP port 80 by default.
- HTTP is a stateless request / response protocol.
- HTTP is text oriented
- Typical methods are
  - GET: Retrieval of the document identified in the URL
  - HEAD: Retrieval of meta information about the document identified in the URL
  - ...
- An HTTP server only needs to implement the GET and HEAD methods.
- Design goals of HTTP are:
  - **Simplicity:** The protocol should be easy to implement.
  - **Efficiency:** The protocol should only consume a few resources (We are currently moving away from this.)
  - **Performance:** The protocol should be as fast as possible.

# Tim Berners-Lee



Tim Berners-Lee was knighted by Queen Elizabeth for his invention of the World Wide Web. He is shown here, along with the first picture posted on the Web and a screen shot from an early version of his Web browser.

# HTTP is an application layer protocol



- ▶ The Web client and the Web server are application programs
- ▶ Application layer programs do useful work like retrieving Web pages, sending and receiving email or transferring files
- ▶ Lower layers take care of the communication details
- ▶ The client and server send messages and data without knowing anything about the communication network

# The application layer is boss - the top layer

| Layer        | Function                                                   |
|--------------|------------------------------------------------------------|
| Application  | Do useful work like Web browsing, email, and file transfer |
| Lower layers | Handle communication between the client and server         |

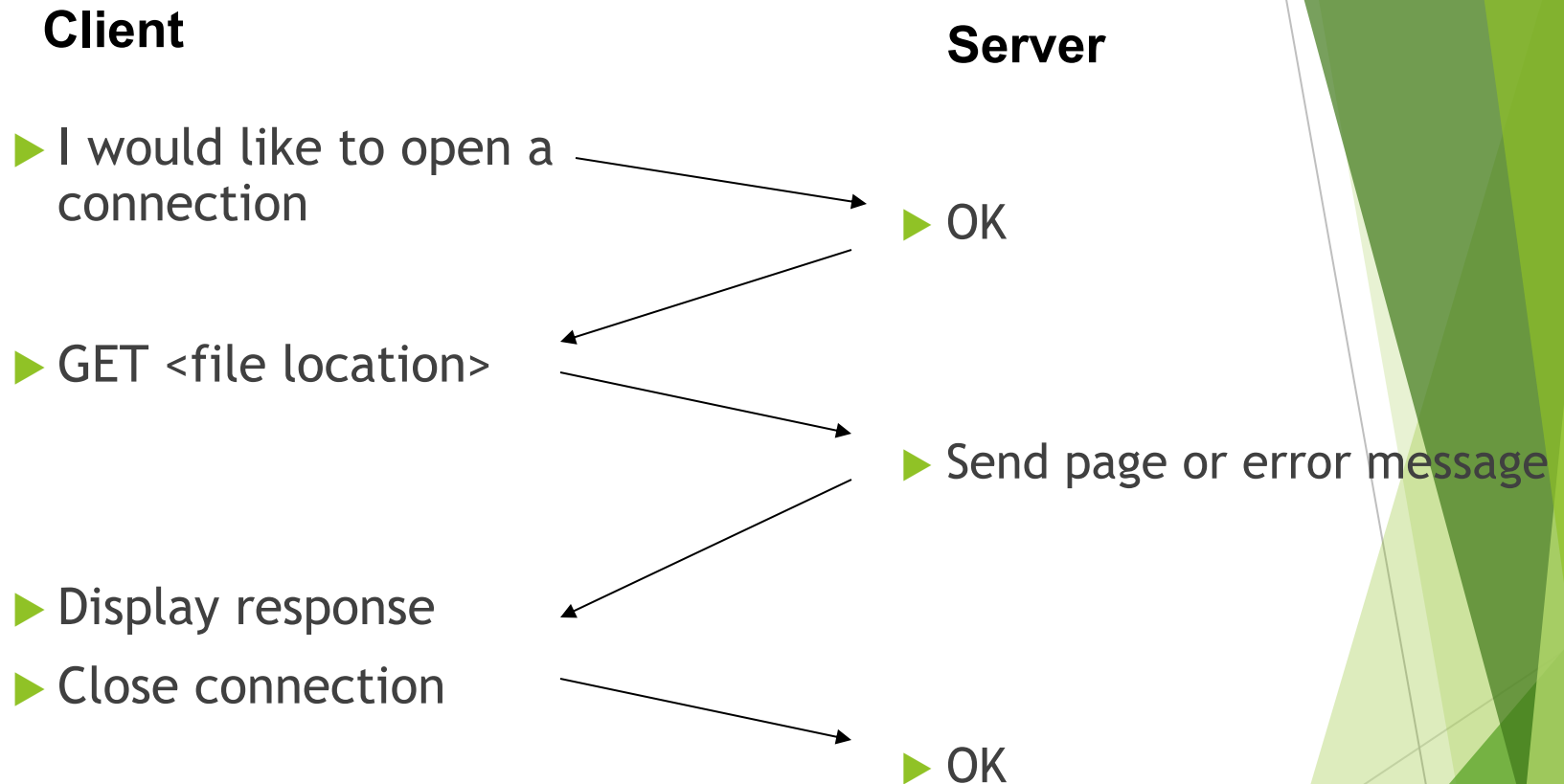
► **Your boss says:** Send this package to Miami -- I don't care if you use Federal Express, UPS, or any other means. Also, let me know when it arrives or if it cannot be delivered for some reason.

► **The application program says:** Send this request to the server -- I don't care how you do it or whether it goes over phone lines, radio, or anything else about the details. Just send the message, and let me know when it arrives or if it cannot be delivered for some reason.

Many application layer protocols are used on the Internet, HTTP is only one

| Protocol                    | Application                                               |
|-----------------------------|-----------------------------------------------------------|
| HTTP: Hypertext Transfer    | Retrieve and view Web pages                               |
| FTP: File Transfer          | Copy files from client to server or from server to client |
| SMTP: Simple Mail Transport | Send email                                                |
| POP: Post Office            | Read email                                                |

# An HTTP conversation

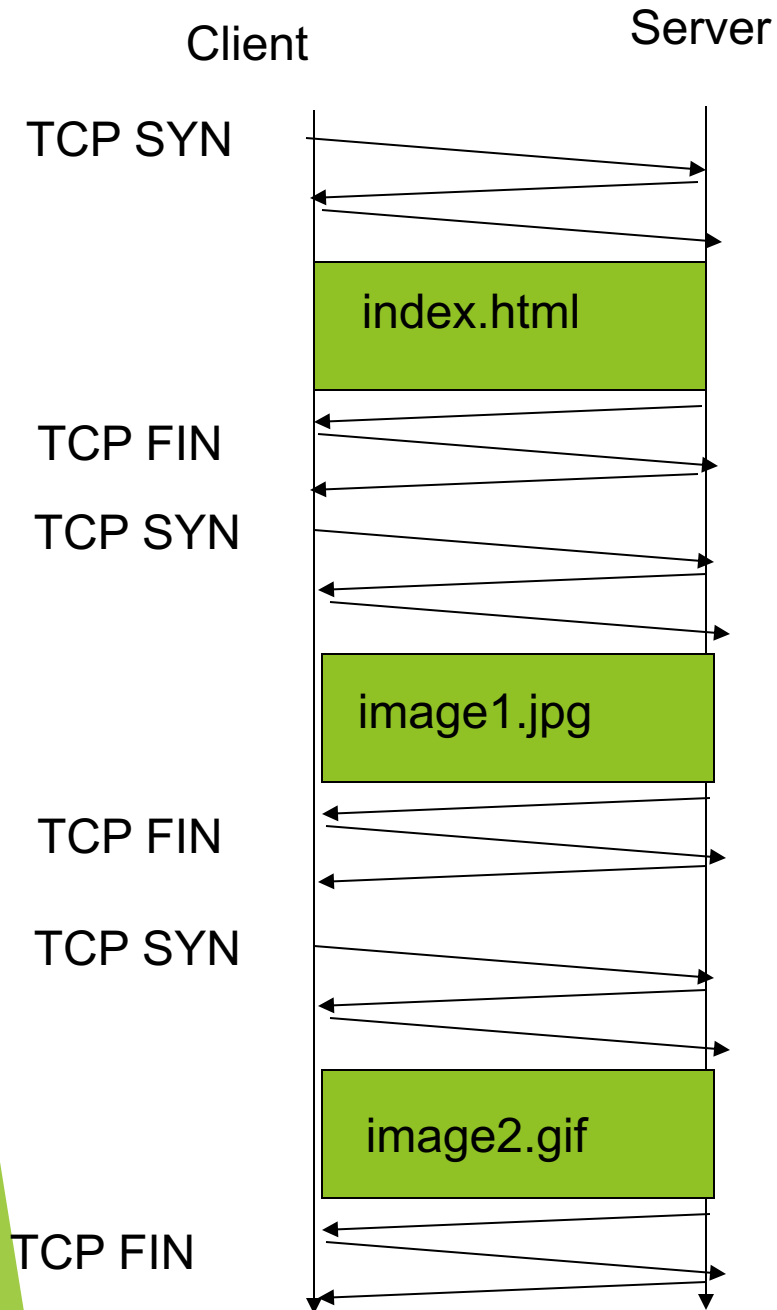


HTTP is the set of rules governing the format and content of the conversation between a Web client and server

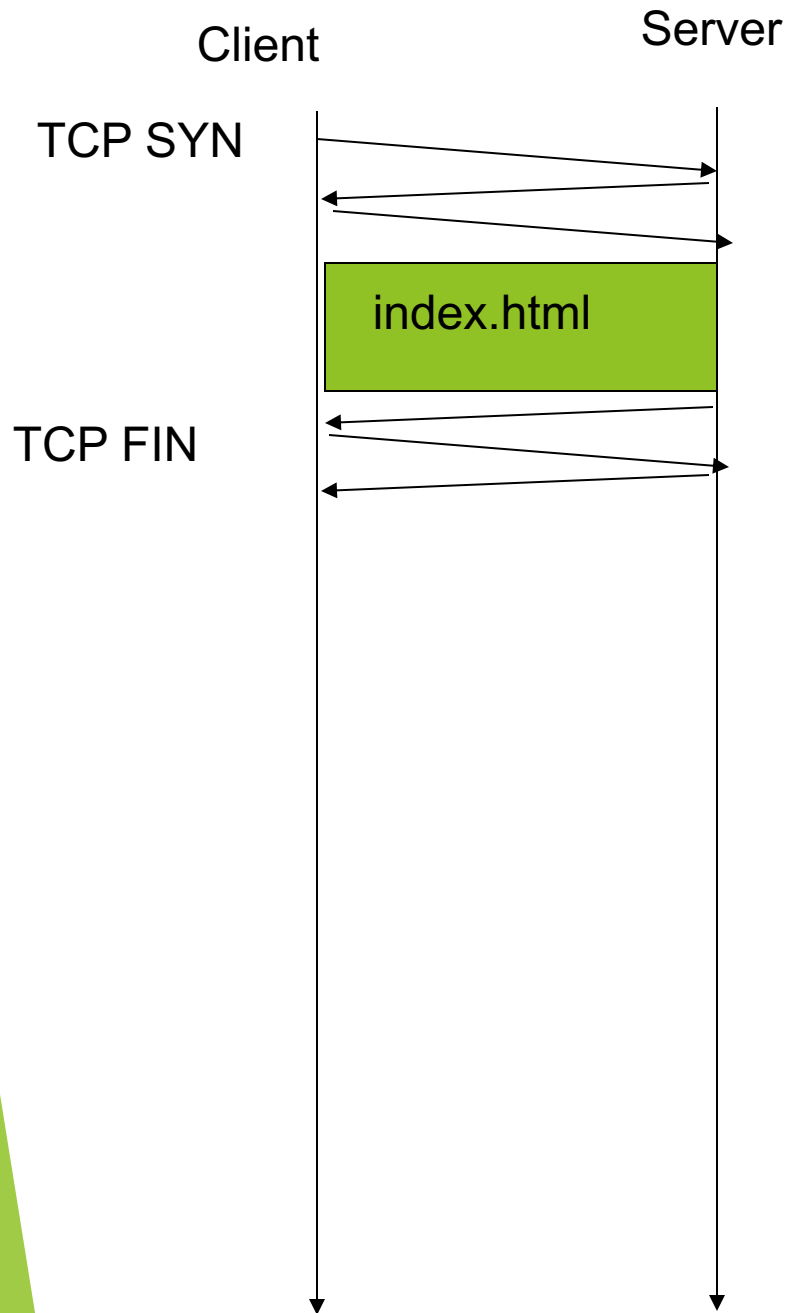
# Possible Communication Models for HTTP

1. The “classic” approach: HTTP/1.0
2. HTTP over multiple, concurrent TCP connections
3. Persistent-HTTP: HTTP/1.1
4. HTTP Pipelining
5. Transaction-TCP (T/TCP)
6. HTTP over UDP-based protocols

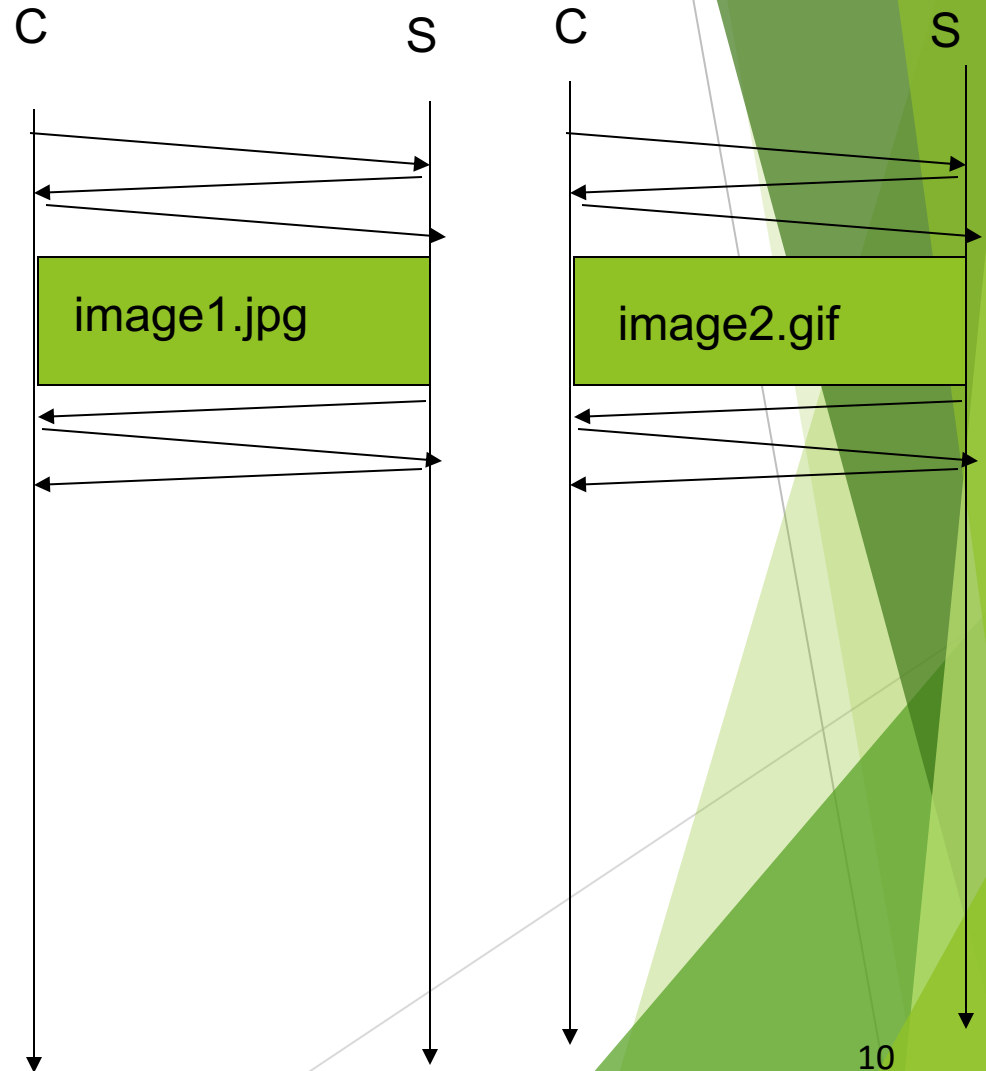


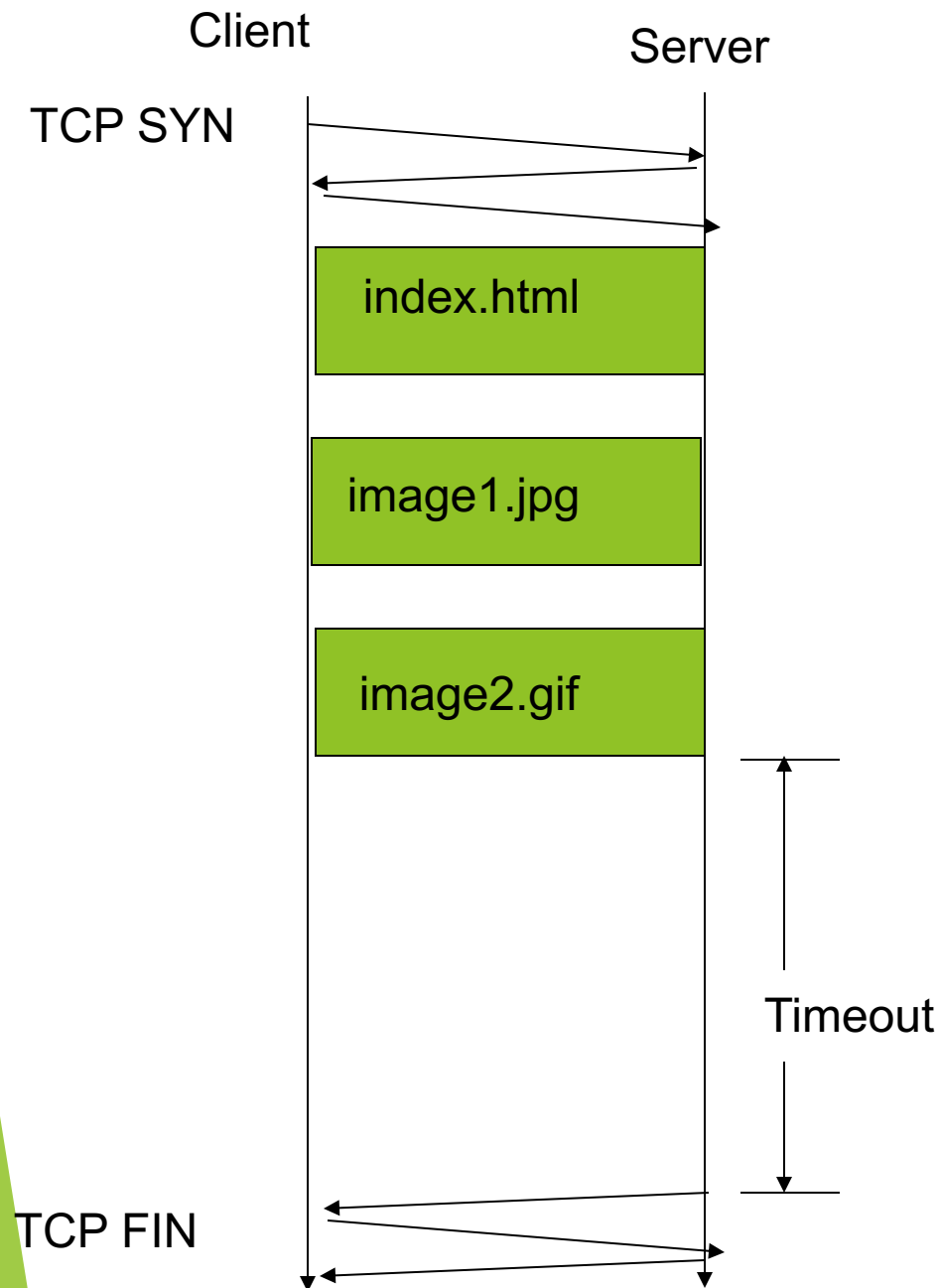


The “classic” approach in HTTP/1.0 is to use one HTTP request per TCP connection, serially.

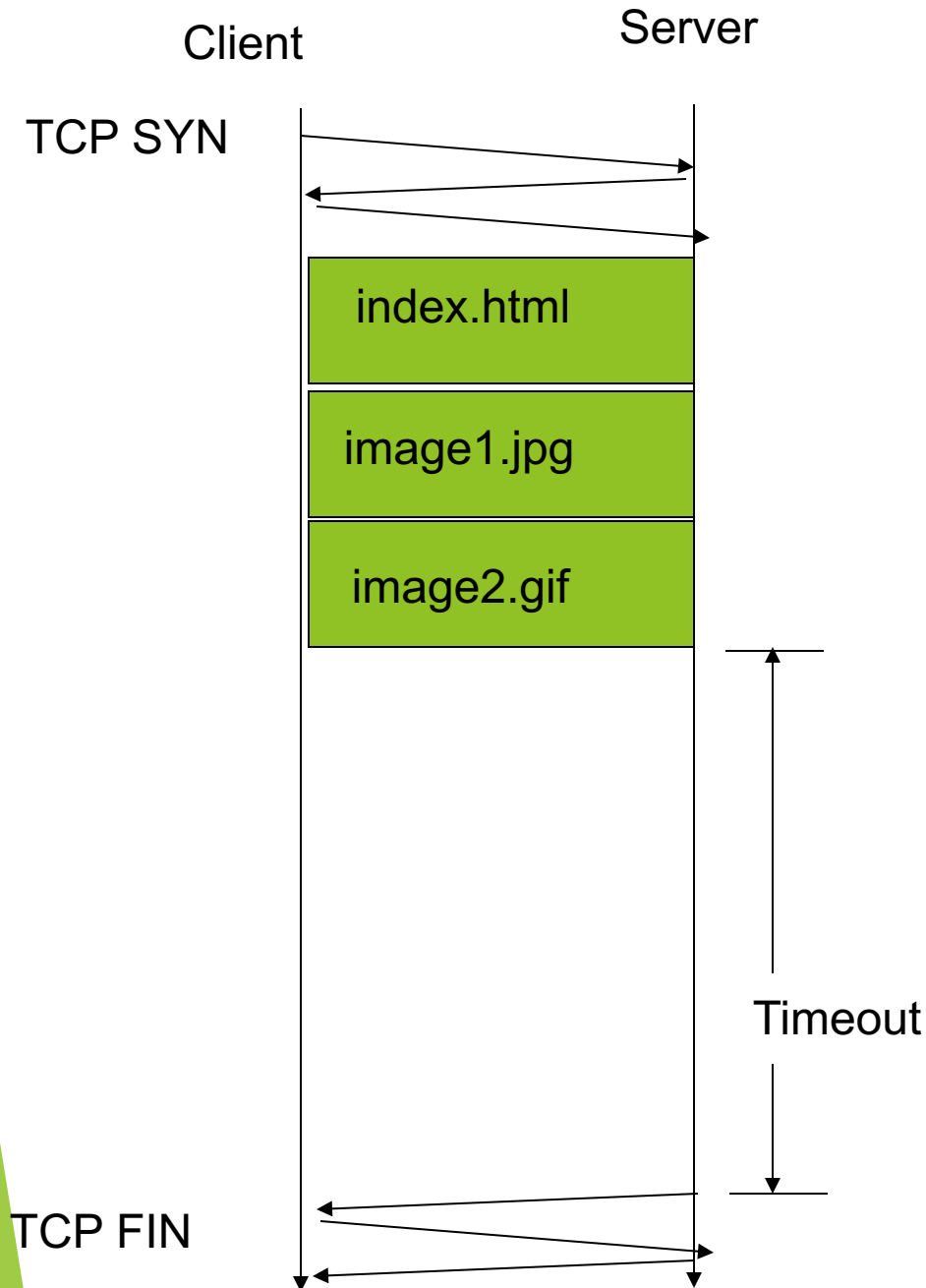


Concurrent (parallel) TCP connections can be used to make things faster.





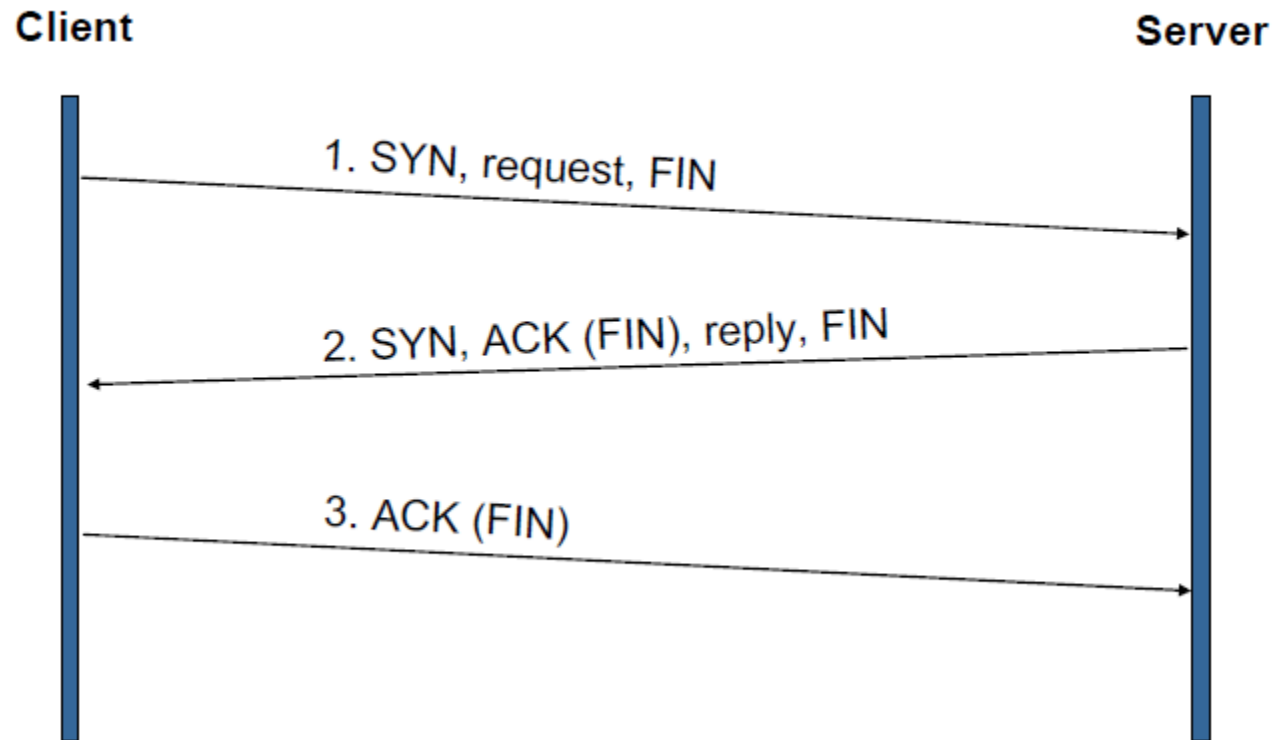
The “persistent HTTP” approach can re-use the same TCP connection for Multiple HTTP transfers, one after another, serially. Amortizes TCP overhead, but maintains TCP state longer at server.



The “pipelining” feature in HTTP/1.1 allows requests to be issued asynchronously on a persistent connection. Requests must be processed in proper order. Can do clever packaging.

Problem: Head of line blocking

# Transaction-TCP



# Performance Effects of HTTP/1.1

|                               | HTTP/1.0 | HTTP/1.1 Persistent | HTTP/1.1 Pipeline |
|-------------------------------|----------|---------------------|-------------------|
| Max simultaneous sockets      | 6        | 1                   | 1                 |
| Total number of sockets used  | 40       | 1                   | 1                 |
| Packets from client to server | 226      | 70                  | 25                |
| Packets from server to client | 271      | 153                 | 58                |
| Total number of packets       | 497      | 223                 | 83                |
| Total elapsed time [secs]     | 1.85     | 4.13                | 3.02              |

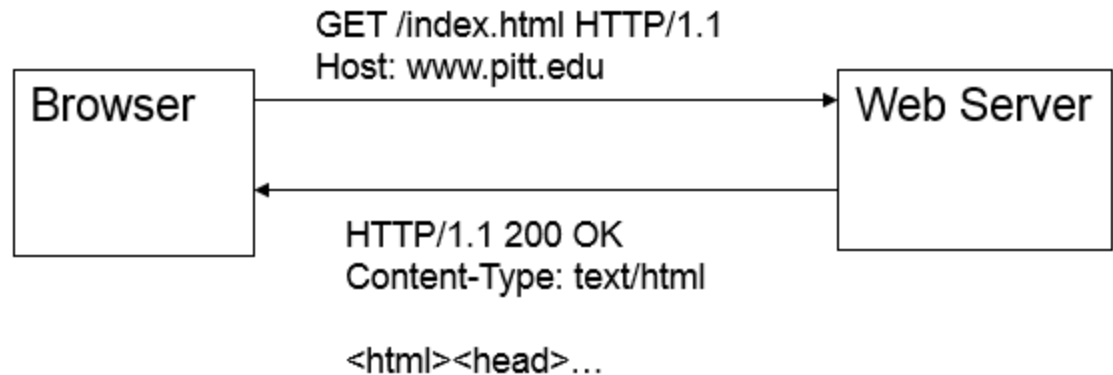
Nielsen, Hendrik Frystyk, Gettys, Jim, Baird-Smith, Anselm, Prud'hommeaux, Eric, Håkon Wium Lie, Lilley, Chris: Network Performance Effects of HTTP/1.1, CSS1, and PNG, W3C, NOTE 24-June 1997.

# HTTP over UDP

- What is the motivation?
- What are the advantages?
- What are the disadvantages?
- What are the consequences?

# HTTP Messages (1)

- ▶ HEAD
- ▶ GET
- ▶ POST
- ▶ PUT
- ▶ DELETE
- ▶ TRACE
- ▶ OPTIONS
- ▶ CONNECT
- ▶ PATCH





# HTTP methods (2)

| Method | Description                                                                                                                                                                                                                                                                                  |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GET    | <p>Request for a resource. The resource is specified in the URL. GET is supported by all browsers.</p> <p>Example: GET /index.html</p>                                                                                                                                                       |
| POST   | <p>Used to send data to the server. Main application online forms. The format of the data is described in the Content-Type: attribute. The data is transferred in the body.</p> <p>Example:<br/>Content-Type: application/x-www-form-urlencoded<br/><br/>param1=value1&amp;param2=value2</p> |
| HEAD   | <p>Returns only the headers that would have been used in the case of a GET request. The client can then decide how to proceed.</p> <p>Example: HEAD /downloads/video.mpeg HTTP/1.1</p>                                                                                                       |

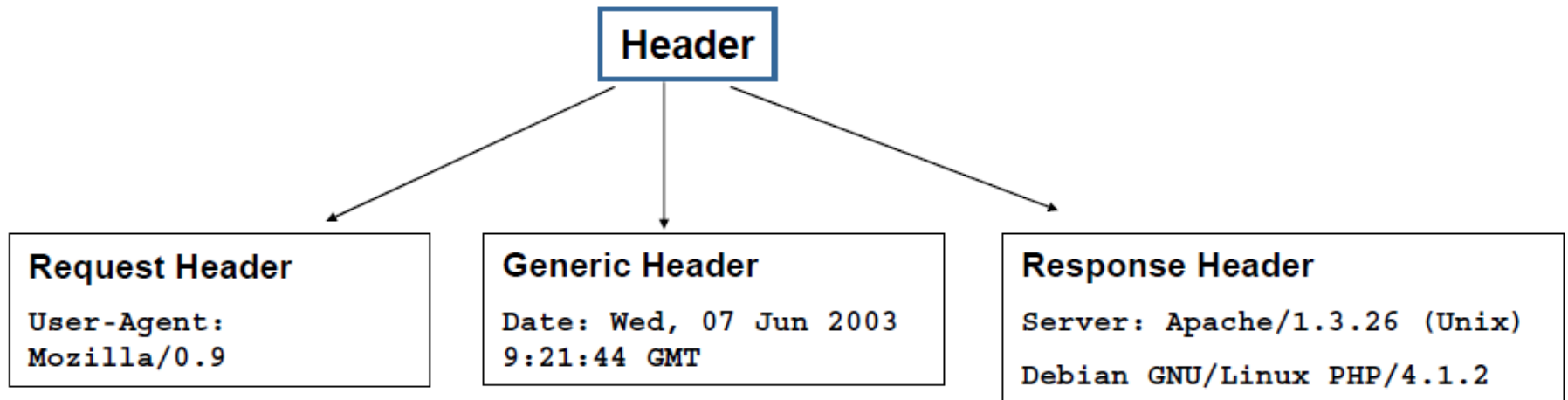
# HTTP methods (3)

| Method  | Description                                                                                                                                                                                                                                                                                                             |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PUT     | <p>Create a new resource or replace an existing one. The data is transferred in the body. Successful answer (code 200 or 204) does not contain a body. The creation of a new resource is indicated by the code 201 (Created).</p> <p>Example: PUT /new.html HTTP/1.1</p>                                                |
| OPTIONS | <p>Asks the server for the permitted HTTP methods. A special URL or the asterisk * can be specified for the entire server. The options can be found in the Response header Allow. The OPTIONS method is often deactivated.</p> <p>Example: OPTIONS <a href="http://bmb.cu.edu.tr">http://bmb.cu.edu.tr</a> HTTP/1.1</p> |
| DELETE  | <p>Deletes the specified resource. Used for API servers. Is switched off on content-oriented servers.</p>                                                                                                                                                                                                               |
| TRACE   | <p>Loop back test. The request is sent back as it was received.</p>                                                                                                                                                                                                                                                     |

# HTTP special methods

| Method  | Description                                                                                                                                                                                                                                                                                                                                                                                                                      |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PATCH   | <p>While the PUT method completely replaces a resource, a resource can be specifically modified with PATCH. The modifications specified in the body must be applied to the resource entirely or not at all ("atomic"). The modifications can, for example, be specified with the correct syntax in the body using a JSON PATCH (RFC6902).</p> <p>The body of the response contains a representation of the changed resource.</p> |
| CONNECT | <p>CONNECT establishes a tunnel to the requested resource. After that, data can be exchanged "blindly" via the tunnel until the tunnel is closed. CONNECT should only be used for requests to a proxy.</p>                                                                                                                                                                                                                       |

# HTTP Messages (2)



# HTTP Messages (3)

**Informational (1xx):** 100 Continue

**Successfull (2xx):** 200 OK

**Redirection (3xx)** 302 Found

**Client Error (4xx)** 400 Bad Request

**Server Error (5xx)** 500 Internal Server Error

# HTTP Response Status Codes

There are a variety of status codes. The most important are:

- **HTTP Status Code 200 – OK**
- **HTTP Status Code 301 - Moved Permanently:**  
Resource has a different URL. This can be found in the location header.
- **HTTP Status Code 302 – Moved Temporarily**
- **HTTP Status Code 403 – Forbidden**
- **HTTP Status Code 404 - Not Found:**  
Forwarding to an error page or to a most relevant page. The redirect to the homepage is bad, as it seems confusing.
- **HTTP Status Code 500 – Internal Server Error:**  
Server crashed.
- **HTTP Status Code 503 - Service Unavailable:**  
Server is overloaded. It is helpful here to provide information about when the server is likely to be available again.

# HTTP Messages - Example

ousystem@ux-2s02 /mnt/www/ousystem 5# telnet www2 80

Trying 194.95.66.20...

Connected to ux-2s05.inf.fh-rhein-sieg.de.

Escape character is '^['.

**GET / HTTP/1.0**

**HTTP/1.1 200 OK**

**Date: Fri, 05 Dec 2003 10:09:52 GMT**

**Server: Apache/1.3.26 (Unix) Debian GNU/Linux PHP/4.1.2 mod\_ssl/2.8.9 OpenSSL/0.9.6g**

**Last-Modified: Wed, 27 Feb 2002 15:12:43 GMT**

**ETag: "b42d-45f-3c7cf76b"**

**Accept-Ranges: bytes**

**Content-Length: 1119**

**Connection: close**

**Content-Type: text/html; charset=iso-8859-1**

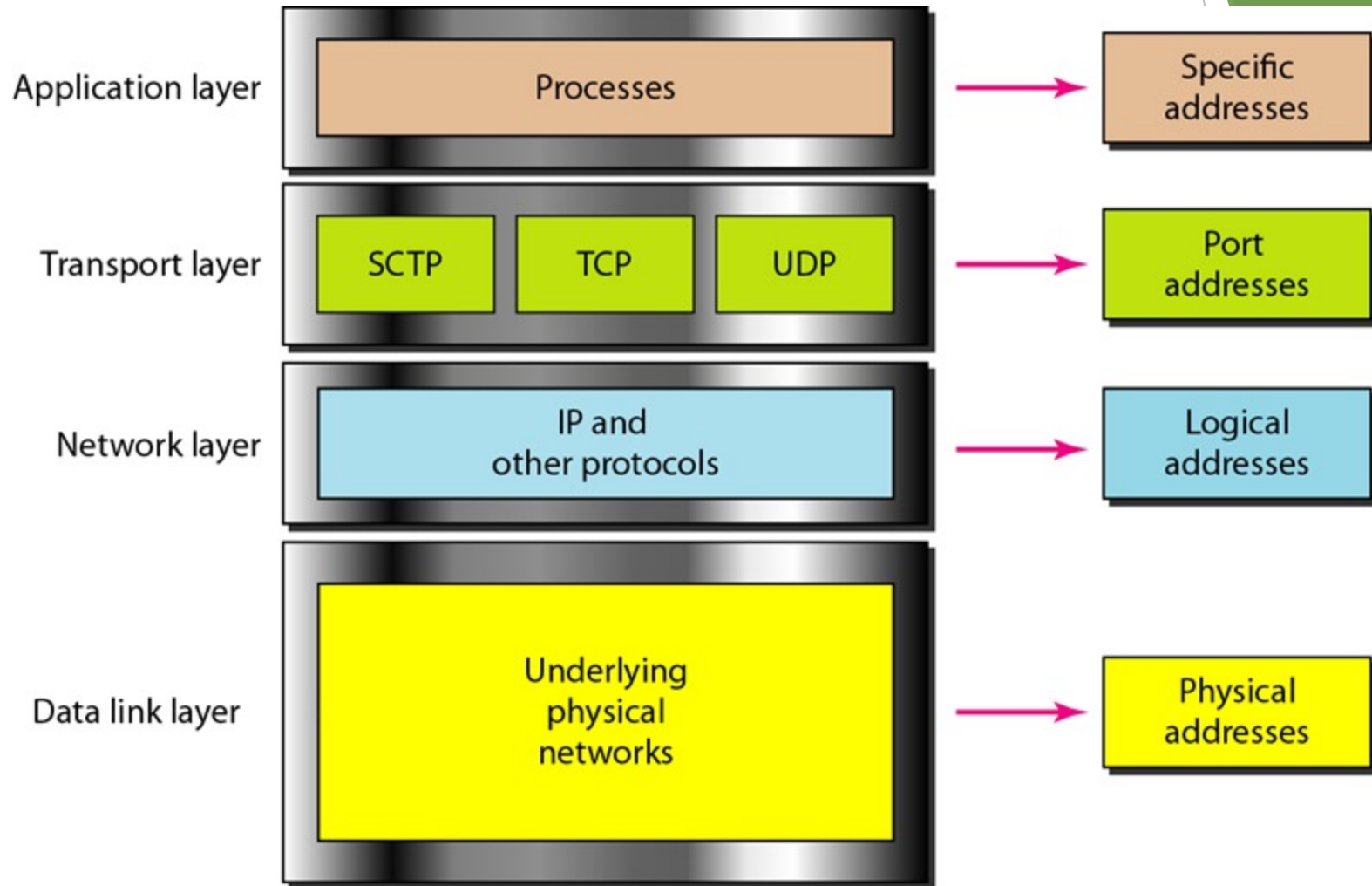
**<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 3.2//EN">**

**<HTML>**

**<HEAD>**

# Relationship of layers and addresses in TCP/IP

HTTP Addressing?





# Identification of resources

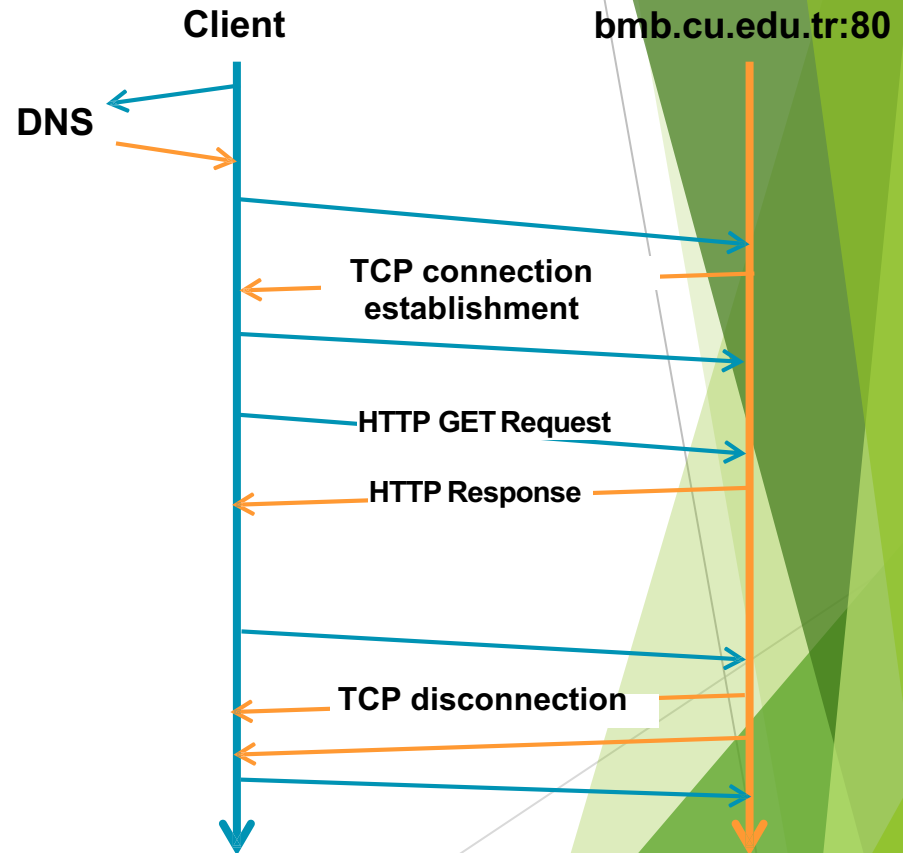
`http://www.example.com:80/path/to/myfile.html?key1=value1&key2=value2#SomewhereInTheDocument`

| Component                                            | Name      | Other examples, remark                                          |
|------------------------------------------------------|-----------|-----------------------------------------------------------------|
| http                                                 | Protocol  | https, mailto, ftp, file                                        |
| <a href="http://www.example.com">www.example.com</a> | Authority | 192.34.23.8                                                     |
| 80                                                   | Port      | Can be omitted or other port numbers                            |
| path/to/myfile.html                                  | Path      | Path to the resource                                            |
| key1=value1&key2=value2                              | Query     | Syntax is not fixed, server specific                            |
| SomewhereInTheDocument                               | Fragment  | Pointer for referring to a particular fragment in the resource. |

# Basic sequence of an HTTP request response

Example: Access to <http://bmb.cu.edu.tr:80/>

1. DNS resolution
2. Establishing a TCP connection to 193.140.54.17:80
3. Send HTTP GET request messages as bytestrome to 193.140.54.17:80.
4. Send HTTP response messages as bytestromes to the client.
5. Termination of the TCP connection by the client or server
6. Termination of the TCP connection by the server or client



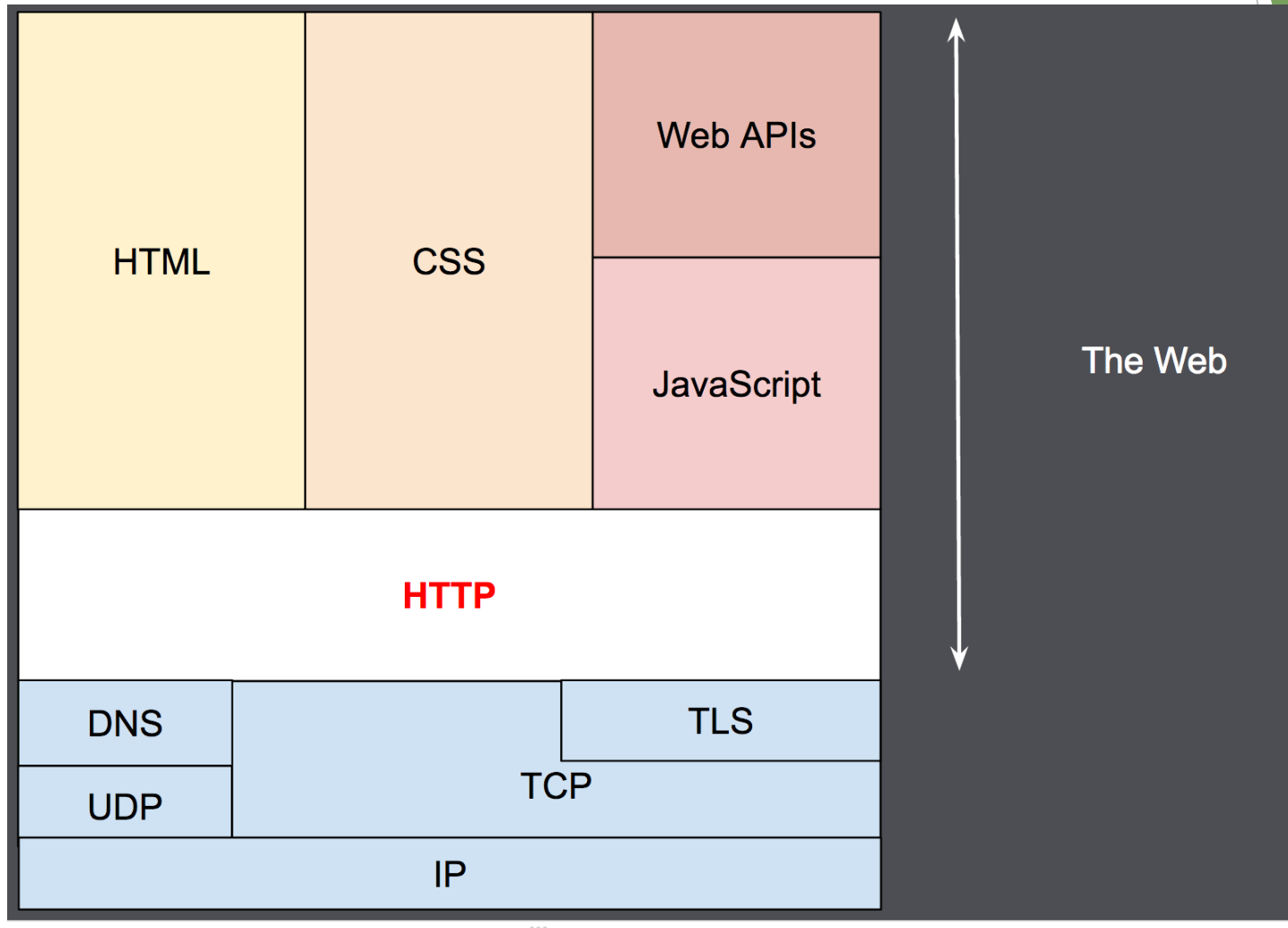
# HTTP History

- ▶ 1991 - **HTTP 0.9** defined, supports only hypertext, linked to port 80 (no images)
  - ▶ Tim Berners-Lee starts World Wide Web project at CERN
  - ▶ Goal: Simplicity, just get an ASCII page from the server
- ▶ 1992 - **HTTP 1.0**, supports images, scripts as well
  - ▶ Goal: Transfer of any file format between client and server
- ▶ 1994 - CERN and MIT agree to set up WWW Consortium
- ▶ 1997 - **HTTP 1.1**, supports open ended extensions
  - ▶ Goal: Performance optimization, caching, pipelining.
- ▶ 2000 - Roy Fielding: Doctoral thesis "The REST-API was born (REST = REpresentational State Transfer)
- ▶ 2012 - First HTTP 2.0 based on SPDY (Google) HTTP 2.0. RFC 7540.
- ▶ 2015 - Concept: Performance optimization through multiplexing (parallel requests), stream prioritization, server push, "must" TLS.
- ▶ 2018 - Hypertext Transfer Protocol (HTTP) over QUIC UDP-based HTTP
- ▶ 2019 - HTTP-over-QUIC goes over to HTTP/3.

# HTTP vs HTML

- ▶ HTML: hypertext **markup language**
  - ▶ Definitions of tags that are added to Web documents to control their appearance
- ▶ HTTP: hypertext transfer **protocol**
  - ▶ The rules governing the conversation between a Web client and a Web server

# Web technologies and OSI



# References

IETF: *RFC 2616 - Hypertext Transfer Protocol -- HTTP/1.1*,  
<https://datatracker.ietf.org/doc/html/rfc2616>, June 1999.

IETF: *RFC 7230 - Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing*, <https://datatracker.ietf.org/doc/html/rfc7230>, June 2014.

IETF: *RFC 7231 - Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content*,  
<https://datatracker.ietf.org/doc/html/rfc7231>, June 2014.

Mozilla and individual contributors: *HTTP*, <https://developer.mozilla.org/en-US/docs/Web/HTTP> (abgerufen am 13.05.2021)

David Gourley, Brian Totty, Marjorie Sayer, Anshu Aggarwal, Sailu Reddy: *HTTP: The Definitive Guide*, O'Reilly Media, Inc., 2002